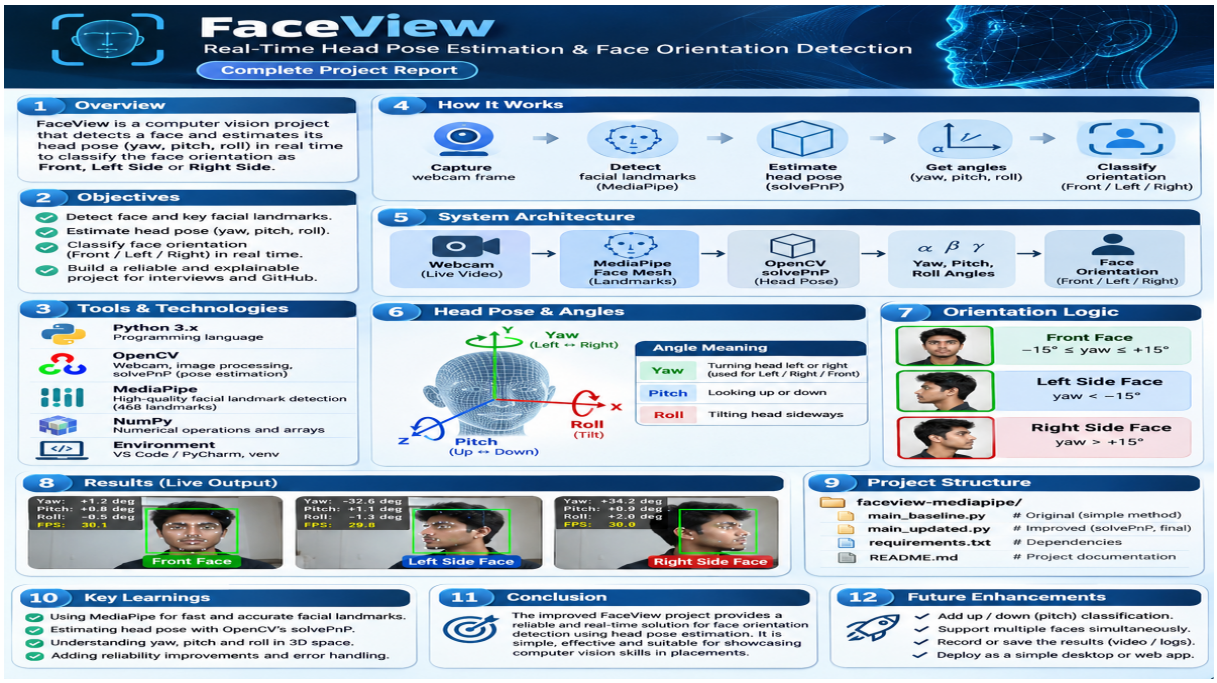
FACEVIEW

Multi-Angle Face Orientation Detection

Final learning guide • Basic vs updated implementation • Code concepts • Interview preparation



Purpose: Understand the original approach, why it was improved, how the pose-estimation pipeline works, and how to explain the project honestly in interviews.

Accuracy note: This guide documents the recorded design and project state. Code snippets are teaching examples unless explicitly described as the recorded baseline logic. Confirm exact line-by-line details against the current source before claiming a full source audit.

1. Project overview

FaceView uses a webcam to estimate whether a person is facing approximately forward or turning to the left or right. It processes live frames, detects facial landmarks, estimates head orientation, and displays a label on the video.

Question	Answer
Input	Live webcam frames.
Landmarks	MediaPipe Face Mesh provides facial landmark coordinates.
Pose estimation	The improved version uses OpenCV solvePnP with paired 3D reference points and 2D image points.
Output	A front/left/right label, estimated angles, and visual overlays when pose estimation succeeds.
Tools	Python, OpenCV, MediaPipe, NumPy, plus math and time in the improved script.

2. The whole pipeline

Step	What happens	Why
1	Open webcam and read a frame.	Get a live image.
2	Flip image horizontally.	Mirror-like preview.
3	Convert BGR to RGB.	MediaPipe expects RGB input.
4	Run Face Mesh.	Get facial landmark coordinates.
5	Convert normalized coordinates to pixels.	Prepare image points for pose estimation.
6	Pair 2D landmarks with 3D reference points.	Connect observed face points to a simple 3D model.
7	Call solvePnP.	Estimate rotation and translation.
8	Convert rotation to angles.	Extract yaw, pitch, roll.
9	Use yaw threshold and draw HUD/axes.	Show the orientation label and feedback.
10	Quit and release resources.	End cleanly.

3. Basic vs updated: side-by-side

Aspect	Basic version: main_baseline.py	Updated version: main_updated.py
Core idea	2D facial symmetry heuristic.	3D-reference-based head-pose estimation.
Main evidence	Horizontal distances from the eyes to the nose.	Corresponding 2D landmark pixels and generic 3D model points.
Decision	If the two distances are similar (difference below the recorded 0.05 threshold), label Front Face; otherwise Side Face.	Use estimated yaw angle and a degree threshold.
Labels	Front Face / Side Face.	Front Face / Left Side Face / Right Side Face.
Angles	No explicit yaw, pitch, or roll calculation.	Estimates yaw, pitch, and roll.
Extra display	Bounding box and face-mesh contours in the recorded baseline description.	Pose axes, angle HUD, and FPS display in the recorded improved design.
Main strength	Simple and easy to understand.	More expressive orientation estimate; separates the two side directions.
Main limitation	A 2D symmetry difference is not a true head-pose angle and can be affected by geometry/position.	Still approximate: generic 3D model, approximate camera matrix, landmark quality, and sign convention affect results.

4. Basic version explained simply

What it measures

The baseline selects landmark 33 and 263 for the eye regions and landmark 1 for the nose. It compares the absolute horizontal eye-to-nose distances. If the two distances are close, the face looks approximately symmetric in the image, so it calls the face front-facing.

```
left_distance  = abs(left_eye_x  - nose_x)
right_distance = abs(right_eye_x - nose_x)

# Conceptual baseline decision
if abs(left_distance - right_distance) < 0.05:
    label = "Front Face"
else:
    label = "Side Face"
```

The snippet is conceptual, not a verbatim copy of the original file. In the recorded baseline, the 0.05 comparison is the symmetry threshold; verify the coordinate scale in the exact source before interpreting that number as pixels or degrees.

Example

Situation	Eye-to-nose distances	Likely baseline label
Looking forward	left = 0.12, right = 0.13; difference = 0.01	Front Face (difference below 0.05)
Turning sideways	left = 0.05, right = 0.19; difference = 0.14	Side Face (difference above 0.05)

These values are made-up examples to demonstrate the comparison, not actual measurements from a test recording.

Why it is limited

It does not calculate a 3D rotation or a yaw angle. A face can look asymmetric because of perspective, landmark noise, face shape, or position. It also groups both left and right turns into one Side Face label.

5. Updated version explained simply

The key change

Instead of asking only “Are the eye-to-nose distances similar?”, the updated version asks: “What 3D head orientation best explains where these selected facial landmarks appear in the image?”

Stage	Plain-English explanation
Landmark IDs	Select six facial points, recorded as [1, 152, 263, 33, 287, 57].
MODEL_POINTS_3D	Defines the corresponding reference points in a generic 3D face layout.
build_camera_matrix()	Builds approximate camera intrinsics using frame width as focal length and image centre as principal point.
solvePnP()	Finds a rotation vector and translation vector that fit the 3D points to the 2D image points.
cv2.Rodrigues()	Converts the rotation vector to a 3 × 3 rotation matrix.
rotation_vec_to_euler()	Extracts pitch/yaw/roll angles from the rotation representation.
Yaw threshold	Uses yaw magnitude and sign to choose front, left, or right.
pose_ok	Tracks whether the current pose estimate succeeded so failed estimates can display N/A instead of false zero angles.

6. Compare the same head turn in both versions

Scenario	Basic version	Updated version
Face forward	Looks for similar eye-to-nose distances → Front Face.	If |yaw| is below 20° → Front Face.
Turn to one side	Distances become less similar → Side Face; direction is not identified.	Yaw sign and threshold → Left Side Face or Right Side Face.
Turn to the other side	Usually still Side Face.	Opposite yaw sign should produce the opposite side label, once sign mapping is verified.
Pose estimation fails	Baseline logic may still make a symmetry decision if its landmarks are available.	pose_ok should remain false; angle HUD should show N/A in the reliability-edited design.

Important: the updated version's left/right sign convention must be verified with the mirrored webcam preview. Do not assume positive yaw always matches the intuitive direction without testing the actual implementation.

7. Imports and constants

Item	Purpose
cv2	Webcam capture, image conversion, solvePnP/Rodrigues, drawing and display.
mediapipe	Face Mesh landmark detection.
numpy	Numeric arrays for points and camera parameters.
math	Angle-related calculations.
time	Timing/FPS calculations.
YAW_THRESHOLD_DEG = 20	Front-facing tolerance in degrees for the updated classifier.
LANDMARK_IDS = [1, 152, 263, 33, 287, 57]	Selected facial landmarks used by the improved pose estimate.

8. The 3D model and 2D image points

```
MODEL_POINTS_3D = np.array([
    [   0.0,    0.0,    0.0],
    [   0.0, -330.0,  -65.0],
    [-225.0,  170.0, -135.0],
    [ 225.0,  170.0, -135.0],
    [-150.0, -150.0, -125.0],
    [ 150.0, -150.0, -125.0],
], dtype=np.float64)
```

Each row represents a point in a generic 3D reference face. MediaPipe provides the corresponding 2D positions in the image. The points must match in the same order; mismatched correspondence can produce a bad pose estimate.

The 3D points are generic, not a personal scan. Their units are model units, and the model's assumptions affect the result.

9. Camera matrix

The camera matrix represents simplified camera-intrinsic parameters. The recorded implementation approximates focal length as the frame width and places the principal point at the image centre.

```
# Conceptual example for a 640 x 480 frame
[
    [640,   0, 320],
    [  0, 640, 240],
    [  0,   0,   1],
]
```

This is not a calibrated camera matrix. Camera calibration could provide more accurate intrinsic parameters.

10. solvePnP, rvec/tvec, and Rodrigues

OpenCV solvePnP takes 3D model points, corresponding 2D image points, a camera matrix, and distortion coefficients. It returns a success flag and estimates of rotation and translation.

```
# Illustrative call, not guaranteed verbatim
success, rvec, tvec = cv2.solvePnP(
    MODEL_POINTS_3D, image_points_2d,
    camera_matrix, distortion_coefficients
)
rotation_matrix, _ = cv2.Rodrigues(rvec)
```

Term	Meaning
success	Whether the pose solver found a solution.
rvec	Rotation vector: the estimated 3D orientation representation.
tvec	Translation vector: estimated position relative to the camera.
Rodrigues	Converts the rotation vector into a rotation matrix.

11. Euler angles and classification

Angle	Simple meaning
Yaw	Turn left or right.
Pitch	Tilt up or down.
Roll	Tilt toward a shoulder.

The improved function converts the rotation representation into pitch, yaw, and roll in degrees. The precise formulas depend on the rotation order and the source implementation; verify the exact function before memorising the equation.

```
# Conceptual classification
if abs(yaw) < 20:
    label = "Front Face"
elif yaw < -20:
    label = "Left Side Face"
else:
    label = "Right Side Face"
```

Example: yaw +8° is front-facing; yaw –28° maps to left; yaw +35° maps to right under the recorded sign convention. The mirrored preview may affect which real-world turn produces each sign.

The threshold is a chosen rule, not a trained classifier. The yaw estimate itself is approximate.

12. pose_ok, HUD, axes, and FPS

pose_ok is a Boolean state for whether the current frame's pose estimate succeeded. It should reset for each current attempt, become true only on success, and drive whether angles or N/A are displayed.

The axis overlay draws coordinate directions near the nose to visualise the estimated orientation. The HUD displays angle estimates and FPS. FPS is an approximate frames-per-second indicator, not a formal benchmark.

A failed pose is not the same as a valid 0° estimate. Showing N/A avoids falsely implying the head was measured as straight ahead.

13. Webcam loop and cleanup

Operation	Meaning
VideoCapture(0)	Open the default camera device.
isOpened()	Check whether the camera opened.
read()	Fetch the next frame.
flip(frame, 1)	Mirror the frame horizontally.
BGR → RGB	Convert OpenCV's channel order to the RGB format expected by MediaPipe.
face_mesh.process()	Run landmark detection.
imshow() / waitKey()	Display the annotated frame and allow keyboard input.
q / release / destroyAllWindows	Quit, release the camera, and close OpenCV windows.

14. Strengths, limitations, and possible improvements

Basic approach	Updated approach
Strength: minimal geometry and easy to explain.	Strength: estimates orientation and can distinguish left/right turns.
Limitation: symmetry is only a rough cue.	Limitation: generic model and approximate intrinsics affect pose quality.
Improvement: use a pose-estimation method if angle/direction is required.	Improvements: calibrate the camera, validate angle signs, test varied lighting/poses, improve failure handling, and quantify accuracy.

15. How to run and test

Run the improved version from the project folder:

```
.\.venv\Scripts\python.exe -u main_updated.py
```

Run the baseline separately for comparison:

```
.\.venv\Scripts\python.exe .\main_baseline.py
```

Only run one version at a time because they share the webcam. Press q in the video window to quit.

Test	Expected check
Face forward	Front Face when yaw is within the threshold / baseline distances are sufficiently similar.
Turn left/right	Updated labels should respond consistently; verify sign mapping.
Landmarks/pose fail	Updated HUD should show N/A when pose estimation fails.
Camera unavailable	Open-camera check should produce a useful error.
Quit	Camera and windows should be released.

16. Interview-ready explanation

> “FaceView is a real-time head-orientation project built using Python, OpenCV, and MediaPipe. The original version used a simple 2D symmetry heuristic by comparing eye-to-nose distances to classify the face as front-facing or side-facing. I then improved it by matching selected MediaPipe landmarks with a generic 3D face model and using OpenCV solvePnP to estimate head pose. The rotation is converted into yaw, pitch, and roll, and yaw with a threshold is used to classify front, left, or right orientation. The output is shown on the live webcam feed. The main limitations are the generic 3D model, approximate camera intrinsics, and dependence on landmark quality.”

17. Interview questions and short answers

Why did you improve the baseline?

The symmetry heuristic only gave front/side and did not estimate a pose angle or separate left from right.

What does MediaPipe do?

Detects facial landmarks in the frame.

What does solvePnP do?

Estimates rotation and translation from matched 3D reference points and 2D image points.

Why do the 2D and 3D points need to match in order?

Each image point must correspond to the same facial feature as its reference 3D point.

What is yaw?

The left/right rotation component under the chosen angle convention.

Why use a yaw threshold?

To tolerate small deviations from straight ahead before classifying a side turn.

Why show N/A?

A failed estimate must not be mistaken for a valid zero-degree measurement.

Does this train a custom model?

The described pipeline uses MediaPipe's existing landmark detector and geometric pose estimation; it does not describe training a custom model.

What would you improve?

Calibrate the camera, validate left/right sign mapping, test under varied conditions, and measure performance/accuracy systematically.

What is the biggest limitation?

The estimate depends on landmarks, generic model assumptions, approximate camera parameters, and threshold choice.

18. Glossary and final revision

Term	Meaning
Frame	One image in a video stream.
Landmark	A facial feature coordinate.
Normalized coordinate	Coordinate scaled relative to image size.
Pixel coordinate	Coordinate measured in image pixels.
Camera matrix	Simplified camera intrinsic parameters.
solvePnP	Estimate pose from corresponding 3D and 2D points.
rvec / tvec	Rotation and translation estimates.
Euler angles	Orientation expressed as three angles.
FPS	Frames processed per second.
Threshold	A cutoff used for a classification decision.
pose_ok	Whether the current pose estimate succeeded.

Revision in one line: Baseline = compare eye/nose symmetry → front or side. Updated = landmarks + 3D reference model + camera matrix + solvePnP → rotation → yaw/pitch/roll → threshold → front/left/right.

Honest status: This guide now includes a dedicated comparison and worked examples. Exact line-by-line source verification and live validation of every failure path still require checking against the actual current local files.